	Politechnika Bydgoska im. J. J. Śniadeckich Wydział Telekomunikacji, Informatyki i Elektrotechniki		
Przedmiot	Zaawansowane programowanie obiektowe		
Prowadzący	dr inż. Damian Szczegielniak		
Temat	Zabezpieczenie usług typu REST przy użyciu tokenów JWT z globalną obsługą błędów i serializacją przy użyciu PagedModel		
Student	Jakub Ziółkowski, grupa 1, Informatyka stosowana, IV semestr, studia stacjonarne		
Nr ćw.	7	Data wykonania	18.07.2026 r.
Ocena		Data oddania spr.	11.08.2026 r.

Spis treści

1. Cel laboratorium.....	2
2. Rozwiązanie	2
2.1. Architektura zabezpieczenia REST API przy użyciu Spring Security i tokenów JWT	2
2.2. Rozszerzenie modelu danych, persystencji i inicjalizacja ról	4
2.3. Obsługa walidacji danych i błędów w REST API.....	6
2.4. Implementacja mechanizmów JWT - JwtService, JwtAuthFilter oraz obsługa uwierzytelniania.....	8
2.5. Serializacja stronicowanych wyników z PagedModel oraz obsługa autoryzacji w Swagger UI	13
2.6. Testowanie zabezpieczonej usługi REST API w programie Postman oraz w Swagger UI.....	14
3. Wnioski końcowe/Podsumowanie.....	18

1. Cel laboratorium

W ramach realizacji siódmej instrukcji laboratoryjnej należało zaimplementować mechanizm zabezpieczeń dla usługi sieciowej typu REST API. Wykorzystałem do tego celu architekturę opartą na tokenach JWT (JSON Web Token), zintegrowaną z modułem Spring Security frameworka Spring Boot (wersja 4.1.0). Równolegle celem było wdrożenie globalnej obsługi błędów i walidacji danych wejściowych za pomocą adnotacji `@RestControllerAdvice`, jak również optymalizacja serializacji danych stronicowanych do formatu JSON przy użyciu `PagedModel` z biblioteki Spring Data.

Założeniem była zamiana uwierzytelniania opartego na sesjach HTTP oraz mechanizmu HTTP Basic Authentication na wersję w pełni bezstanowej. W następstwie tej modyfikacji tożsamość użytkownika oraz jego uprawnienia (role) są poświadczane podpisanym kryptograficznie tokenem dostępowym oraz odświeżającym.

W zadaniu zmodyfikowałem model bazy danych w PostgreSQL, zastosowałem szyfrowanie haseł algorytmem BCrypt, automatyczną inicjalizację ról systemowych, a także filtr sieciowy do weryfikacji nagłówków i zapewniłem wsparcie dla nagłówków autoryzacyjnych w interfejsie OpenAPI / Swagger UI.

2. Rozwiązanie

2.1. Architektura zabezpieczenia REST API przy użyciu Spring Security i tokenów JWT

Architektura zabezpieczeń w projekcie `project-rest-api-jwt` opiera się na tokenach JWT.

We wcześniejszej wersji zabezpieczeń stan sesji był przechowywany w pamięci serwera (`SessionCreationPolicy.STATELESS`). W nowej wersji programu dostęp do chronionych zasobów REST API jest przyznawany na podstawie odpowiednio podpisanego tokena, który klient (czyli np. aplikacja mobilna, front-end w React/Angular lub program Postman) dołącza do każdego żądania HTTP w nagłówku `'Authorization'` jako Bearer Token.

Architektura ta składa się z następujących komponentów:

1. Użytkownicy i role: Encja `Student` implementuje interfejs `UserDetails` ze Spring Security. Relacja `@ManyToMany` z nową encją `Role` reprezentuje uprawnienia (np. `ROLE_USER`, `ROLE_ADMIN`). Hasła są zabezpieczone funkcją skrótu z solą za pomocą `BCryptPasswordEncoder`.

2. Serwis kryptograficzny (`JwtService`): Odpowiada za generowanie, cyfrowe podpisywanie oraz parsowanie tokenów dostępowych (`Access Token`) i odświeżających (`Refresh Token`).

3. Filtr autoryzacyjny (JwtAuthFilter): Przechwytuje każde przychodzące żądanie HTTP, ekstrahuje ciąg Bearer z nagłówka Authorization, weryfikuje podpis i ważność tokena, po czym rejestruje obiekt UsernamePasswordAuthenticationToken w SecurityContextHolder.

4. Serwis uwierzytelniania (AuthService) i kontroler (AuthController): Udostępniają publiczne endpointy /api/register, /api/login oraz /api/refresh do rejestracji nowych kont, logowania i wymiany wygasłych tokenów dostępowych.

5. Konfiguracja łańcucha filtrów (SecurityWebConfig): Definiuje zasady dostępu oparte na rolach, wyłącza ochronę CSRF (z uwagi na bezstanowość) oraz dołącza JwtAuthFilter przed UsernamePasswordAuthenticationFilter.

Pierwszym krokiem realizacji zadania było uzupełnienie pliku build.gradle o biblioteki Java JWT (JJWT) z grupy io.jsonwebtoken: jjwt-api, jjwt-impl oraz jjwt-jackson. Oczywiście, wsparcie dla adnotacji biblioteki Lombok jest także w tym pliku.

Plik konfiguracyjny application.properties rozszerzyłem o klucze kryptograficzne oraz parametry czasu ważności dla tokenów JWT. Klucze zdefiniowałem w parametrach jwt.access-secret-key oraz jwt.refresh-secret-key. Czasy ważności ustawiłem na 15 minut dla tokenu dostępowego oraz 480 minut dla tokenu odświeżania. Ponownie aktywowałem też obsługę wątków wirtualnych.

```
application.properties x
1 spring.application.name=project-rest-api
2 # Spring DataSource
3 spring.datasource.url=jdbc:postgresql://localhost:5432/projekty
4 spring.datasource.username=postgres
5 spring.datasource.password=postgres
6 spring.datasource.driver-class-name=org.postgresql.Driver
7 # Spring JPA
8 # The SQL dialect makes Hibernate generate better SQL for the chosen database # (Postgres 9.5 and later)
9 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
10 # Validation or export of schema DDL to the database (create, create-drop, validate, update, none)
11 spring.jpa.hibernate.ddl-auto=update
12 # Logging JPA Queries
13 spring.jpa.show-sql=true
14 spring.jpa.properties.hibernate.format_sql=true
15 # Spring Security
16 # HTTP authentication credentials
17 spring.security.user.name=admin
18 spring.security.user.password=admin
19
20 spring.threads.virtual.enabled=true
21
22 # Testowy klucz można wygenerować np. za pomocą https://acte.ltd/Utils/randomkeygen (jako Encryption key 256-bit)
23 jwt.access-secret-key=Xn2r5u8x/A?D(G-KaPdSgVkyP3s6v9y$
24 jwt.access-token-validity-in-min=15
25 jwt.refresh-secret-key=2s5v8y/B?E(G+KbPeShVmYq3t6w9z$C&
26 jwt.refresh-token-validity-in-min=480
```

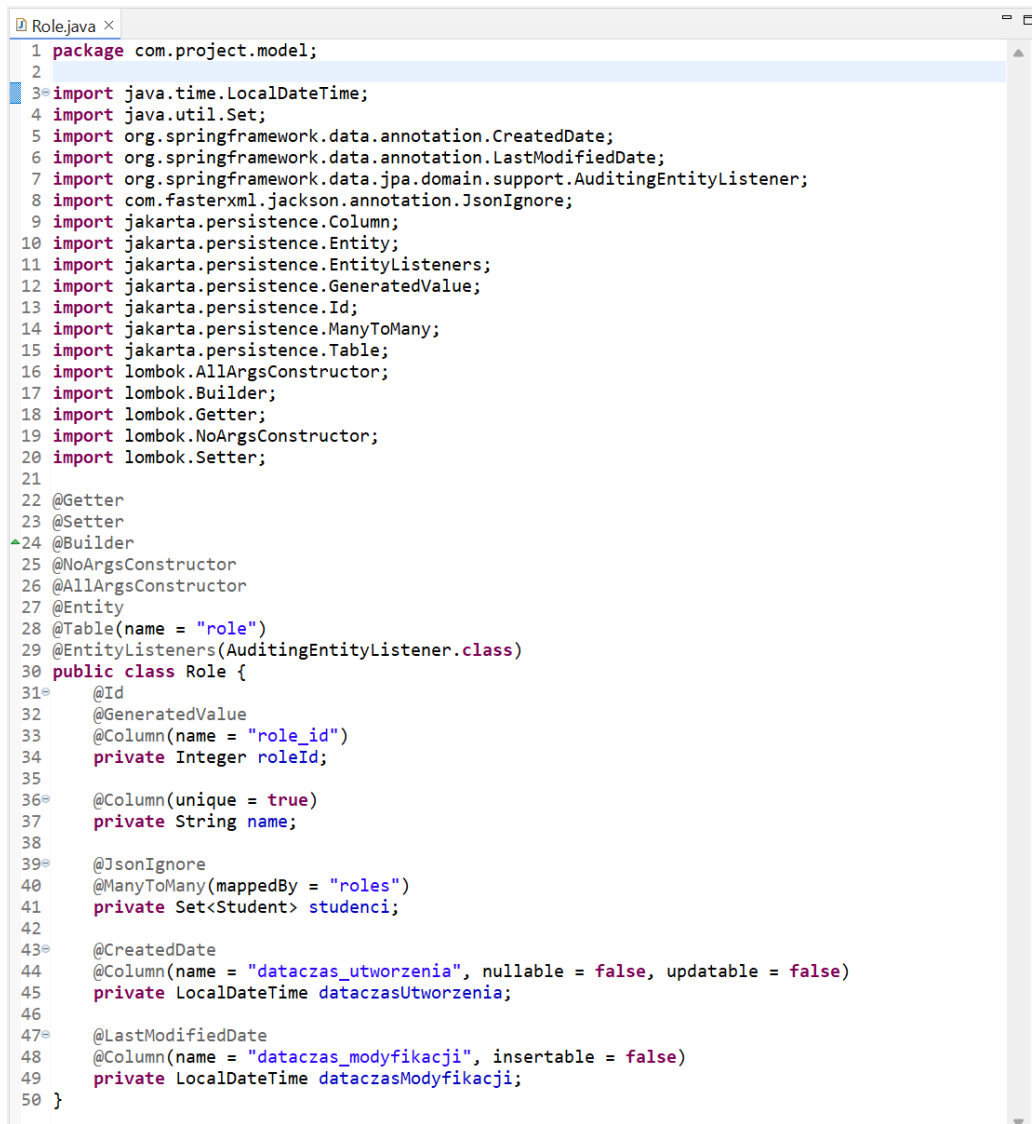
Screen 1 - Zawartość pliku application.properties w środowisku Eclipse IDE

2.2. Rozszerzenie modelu danych, persystencji i inicjalizacja ról

Mechanizm Spring Data JPA Auditing pozwala na automatyczne śledzenie czasu tworzenia oraz modyfikacji rekordów w bazie danych.

Klasa konfiguracyjna JpaConfig w pakiecie com.project.config została opatrzona adnotacją @EnableJpaAuditing. Encje zawierające pola z adnotacjami @CreatedDate i @LastModifiedDate oraz adnotację @EntityListeners(AuditingEntityListener.class) automatycznie uzupełniają znaczniki czasu podczas zapisywania oraz aktualizacji.

Z kolei zabezpieczenie oparte na rolach polega na tym, że w pakiecie com.project.model utworzyłem nową encję Role, która odpowiada tabeli 'role' w bazie danych. Nowa encja przechowuje nazwę roli (np. USER, ADMIN) w kolumnie 'name' oraz znaczniki czasu do audytu JPA.



```
1 package com.project.model;
2
3 import java.time.LocalDateTime;
4 import java.util.Set;
5 import org.springframework.data.annotation.CreatedDate;
6 import org.springframework.data.annotation.LastModifiedDate;
7 import org.springframework.data.jpa.domain.support.AuditingEntityListener;
8 import com.fasterxml.jackson.annotation.JsonIgnore;
9 import jakarta.persistence.Column;
10 import jakarta.persistence.Entity;
11 import jakarta.persistence.EntityListeners;
12 import jakarta.persistence.GeneratedValue;
13 import jakarta.persistence.Id;
14 import jakarta.persistence.ManyToMany;
15 import jakarta.persistence.Table;
16 import lombok.AllArgsConstructor;
17 import lombok.Builder;
18 import lombok.Getter;
19 import lombok.NoArgsConstructor;
20 import lombok.Setter;
21
22 @Getter
23 @Setter
24 @Builder
25 @NoArgsConstructor
26 @AllArgsConstructor
27 @Entity
28 @Table(name = "role")
29 @EntityListeners(AuditingEntityListener.class)
30 public class Role {
31     @Id
32     @GeneratedValue
33     @Column(name = "role_id")
34     private Integer roleId;
35
36     @Column(unique = true)
37     private String name;
38
39     @JsonIgnore
40     @ManyToMany(mappedBy = "roles")
41     private Set<Student> studenci;
42
43     @CreatedDate
44     @Column(name = "dataczas_utworzenia", nullable = false, updatable = false)
45     private LocalDateTime dataczasUtworzenia;
46
47     @LastModifiedDate
48     @Column(name = "dataczas_modyfikacji", insertable = false)
49     private LocalDateTime dataczasModyfikacji;
50 }
```

Screen 2 - Kod encji Role.java w Eclipse IDE

Znacząco zmodyfikowałem klasę encyjną Student, aby pełniła rolę najważniejszego obiektu użytkownika w Spring Security. W tym celu implementuje interfejs UserDetails. Do klasy dodałem też pole password (z adnotacją @JsonProperty(access = Access.WRITE_ONLY), która sprawia, że hash hasła nigdy nie zostanie ujawniony w odpowiedzi JSON) oraz relację @ManyToMany(fetch = FetchType.EAGER) z encją Role przez tabelę łączącą 'student_role'. Zmieniłem oznaczenie pola email na nullable=false oraz unique=true, bo jest loginem użytkownika. W adnotacji @Table są indeksy bazodanowe idx_email, idx_nazwisko oraz idx_nr_indeksu.

```
Student.java x
1 package com.project.model;
2
3 import java.util.Collection;
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37 @Getter
38 @Setter
39 @Builder
40 @NoArgsConstructor
41 @AllArgsConstructor
42 @Entity
43 @Table(name = "student", indexes = {
44     @Index(name = "idx_nazwisko", columnList = "nazwisko", unique = false),
45     @Index(name = "idx_email", columnList = "email", unique = true),
46     @Index(name = "idx_nr_indeksu", columnList = "nr_indeksu", unique = true)
47 })
48 public class Student implements UserDetails {
49
50     @Id
51     @GeneratedValue(strategy = GenerationType.IDENTITY)
52     @Column(name = "student_id")
53     private Integer studentId;
54
55     @NotBlank(message = "Pole imię nie może być puste!")
56     @Size(max = 50, message = "Imię może zawierać maksymalnie {max} znaków!")
57     @Column(nullable = false, length = 50)
58     private String imie;
59
60     @NotBlank(message = "Pole nazwisko nie może być puste!")
61     @Size(max = 100, message = "Nazwisko może zawierać maksymalnie {max} znaków!")
62     @Column(nullable = false, length = 100)
63     private String nazwisko;
64
65     @NotBlank(message = "Pole nr indeksu nie może być puste!")
66     @Size(max = 20, message = "Numer indeksu może zawierać maksymalnie {max} znaków!")
67     @Column(name = "nr_indeksu", nullable = false, unique = true, length = 20)
68     private String nrIndeksu;
69
70     @NotEmpty(message = "Nie podano adresu e-mail")
71     @Email(message = "Niepoprawny format adresu e-mail")
72     @Column(length = 100, nullable = false, unique = true)
73     private String email;
74
75     @JsonProperty(access = Access.WRITE_ONLY)
76     @Size(min = 8, max = 64, message = "Hasło musi składać się z przynajmniej {min} i nie przekraczać {max} znaków")
77     private String password;
78
79     @NotNull(message = "Pole stacjonarny nie może być puste!")
80     @Column(nullable = false)
81     private boolean stacjonarny;
82
83     @ManyToMany(fetch = FetchType.EAGER)
84     @JoinTable(name = "student_role",
85         joinColumns = {@JoinColumn(name = "student_id")},
86         inverseJoinColumns = {@JoinColumn(name = "role_id")})
87     private Set<Role> roles;
88
89     @ManyToMany(mappedBy = "studenci")
90     @JsonIgnoreProperties({"studenci"})
91     private Set<Projekt> projekty;
92
93     @Override
94     public Collection<? extends GrantedAuthority> getAuthorities() {
95         return this.roles
96             .stream()
97             .map(r -> new SimpleGrantedAuthority(r.getName()))
98             .collect(Collectors.toList());
99     }
100
101     @Override
102     public String getUsername() {
103         return this.email; //zakładamy, że e-mail będzie wykorzystywany przy logowaniu
104     }
105 }
```

Screen 3 - Zmodyfikowana encja Student.java z implementacją UserDetails

W warstwie persystencji znajduje się interfejs `RoleRepository` dziedziczący po `JpaRepository<Role, Integer>` i zawierający metodę `Optional<Role> findByName(String name)`.

Do interfejsu `StudentRepository` dodałem metodę `Optional<Student> findByEmail(String email)`, wykorzystywaną podczas procesu logowania i ładowania danych użytkownika.

Proces rejestracji użytkowników oraz ich autoryzacji wymaga obecności ról w bazie danych. Aby nie musieć ręcznie wprowadzać rekordów do PostgreSQL, w nowym pakiecie `com.project.init` umieściłem klasę `DataInitializer`.

Metoda `init()` oznaczona adnotacją `@EventListener(ApplicationReadyEvent.class)` wykonuje się automatycznie po uruchomieniu kontekstu aplikacji Spring Boot, sprawdzając obecność ról 'USER' oraz 'ADMIN' i tworząc je w przypadku ich braku w bazie.

```
DataInitializer.java x
1 package com.project.init;
2
3 import java.util.List;
4 import org.springframework.boot.context.event.ApplicationReadyEvent;
5 import org.springframework.context.event.EventListener;
6 import org.springframework.stereotype.Component;
7 import com.project.model.Role;
8 import com.project.repository.RoleRepository;
9 import lombok.RequiredArgsConstructor;
10
11 @RequiredArgsConstructor
12 @Component
13 public class DataInitializer {
14     private final RoleRepository roleRepository;
15
16     @EventListener(ApplicationReadyEvent.class)
17     public void init() {
18         List.of("USER", "ADMIN")
19             .forEach(role -> roleRepository
20                 .findByName(role)
21                 .orElseGet(() -> roleRepository.save(Role.builder().name(role).build())));
22     }
23 }
```

Screen 4 - Kod klasy `DataInitializer.java` w Eclipse IDE

2.3. Obsługa walidacji danych i błędów w REST API

Zastosowałem scentralizowany mechanizm walidacji danych wejściowych i globalnej obsługi błędów w REST API. W pakiecie `com.project.validation` utworzyłem trzy komponenty:

1. Rekord `Violation`: Przechowuje nazwę błędnego pola (`fieldName`) oraz komunikat błędu (`errorMessage`).

2. Klasa `ValidationService<T>`: Metoda `validate(T object)` waliduje obiekt z użyciem wstrzykniętego `Validator` z pakietu `jakarta.validation`. Jeżeli reguły zostały naruszone, generuje wyjątek `ConstraintViolationException` z wszystkimi informacjami.

3. Klasa `ValidationExceptionHandler`: Zawiera adnotację `@RestControllerAdvice`. Przechwytuje wyjątek `ConstraintViolationException` z dowolnego kontrolera i zamienia go w zbiór obiektów `Violation`, a następnie zwraca klientowi w odpowiedzi kod HTTP 400 Bad Request.

```
ValidationService.java ×
1 package com.project.validation;
2
3 import org.springframework.stereotype.Service;
4 import jakarta.validation.ConstraintViolationException;
5 import jakarta.validation.Validator;
6 import lombok.RequiredArgsConstructor;
7
8 @Service
9 @RequiredArgsConstructor
10 public class ValidationService<T> {
11     private final Validator validator;
12     public void validate(T object) {
13         var violations = validator.validate(object);
14         if (!violations.isEmpty())
15             throw new ConstraintViolationException(violations);
16     }
17 }

ValidationExceptionHandler.java ×
1 package com.project.validation;
2
3 import java.time.format.DateTimeParseException;
4 import java.util.Set;
5 import java.util.stream.Collectors;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.web.bind.annotation.ExceptionHandler;
8 import org.springframework.web.bind.annotation.RestControllerAdvice;
9 import jakarta.validation.ConstraintViolationException;
10
11 @RestControllerAdvice
12 public class ValidationExceptionHandler {
13     @ExceptionHandler(ConstraintViolationException.class)
14     public ResponseEntity<?> handleConstraintValidationException(ConstraintViolationException e) {
15         Set<Violation> violations = e.getConstraintViolations()
16             .stream()
17             .map(v -> new Violation(v.getPropertyPath().toString(), v.getMessage()))
18             .collect(Collectors.toSet());
19         return ResponseEntity
20             .badRequest()
21             .body(violations);
22     }
23 }
```

Screen 5 - Klasy `ValidationService` i `ValidationExceptionHandler` z pakietu `com.project.validation`

Dzięki takiemu układowi w kontrolerach REST nie ma potrzeby powtarzania kodu sprawdzającego wyniki walidacji. Metody w kontrolerach wywołują `validator.validate(obiekt)`, a w przypadku niepoprawnych danych obsługa zostaje przekazana do `ValidationExceptionHandler`.

2.4. Implementacja mechanizmów JWT - JwtService, JwtAuthFilter oraz obsługa uwierzytelniania

W pakiecie `com.project.config` znajduje się `JwtService`. Serwis ten odpowiada za tworzenie, podpisywanie oraz przetwarzanie i weryfikację tokenów JWT.

`JwtService` generuje dwa typy tokenów:

- Access Token: Token dostępowy krótkiego cyklu życia (15 min) przeznaczony do autoryzowania każdego żądania HTTP. Do struktury (Claims) dołączana jest tożsamość użytkownika (Subject - adres email) oraz lista przyznanych ról i uprawnień (authorities).
- Refresh Token: Token odświeżania długiego cyklu życia (480 min) podpisany osobnym kluczem secret key, służący wyłącznie do uzyskiwania nowej pary tokenów bez ponownego podawania hasła.

```

1 package com.project.config;
2
3 import java.nio.charset.StandardCharsets;
4
5 @Service
6 public class JwtService {
7     // W zadaniu tylko dwa klucze służą do generowania i weryfikacji tokenów dla wszystkich
8     // użytkowników, jest to częste rozwiązanie w praktycznych zastosowaniach. Jednak taka para
9     // kluczy może być również dla każdego użytkownika odrębna, unikatowa i przechowywana w bazie.
10    private final SecretKey accessSecretKey;
11    private final Integer accessTokenValidityInMin;
12    private final SecretKey refreshSecretKey;
13    private final Integer refreshTokenValidityInMin;
14
15    // Wartości zdefiniowane w application.properties są automatycznie wstrzykiwane dzięki @Value
16    public JwtService(@Value("${jwt.access-secret-key}") String accessSecretKey,
17                     @Value("${jwt.access-token-validity-in-min}") Integer accessTokenValidityInMin,
18                     @Value("${jwt.refresh-secret-key}") String refreshSecretKey,
19                     @Value("${jwt.refresh-token-validity-in-min}") Integer refreshTokenValidityInMin) {
20        this.accessSecretKey = Keys.hmacShaKeyFor(accessSecretKey.getBytes(StandardCharsets.UTF_8));
21        this.accessTokenValidityInMin = accessTokenValidityInMin;
22        this.refreshSecretKey = Keys.hmacShaKeyFor(refreshSecretKey.getBytes(StandardCharsets.UTF_8));
23        this.refreshTokenValidityInMin = refreshTokenValidityInMin;
24    }
25
26    private String generateToken(Map<String, Object> extraClaims, UserDetails userDetails,
27                                SecretKey secretKey, Integer tokenValidityInMin) {
28        final var authorities = userDetails.getAuthorities()
29            .stream()
30            .map(GrantedAuthority::getAuthority)
31            .toList();
32        final Instant now = Instant.now();
33        final Instant expiration = now.plus(tokenValidityInMin, ChronoUnit.MINUTES);
34
35        return Jwts.builder()
36            .claims(extraClaims)
37            .subject(userDetails.getUsername())
38            .issuedAt(Date.from(now))
39            .expiration(Date.from(expiration))
40            .claim("authorities", authorities)
41            .signWith(secretKey, Jwts.SIG.HS256)
42            .compact();
43    }
44
45    // Claims-y to tzw. deklaracje, będące zbiorem par klucz-wartość
46    public Claims extractAllClaims(String token, SecretKey secretKey) {
47        // Jeżeli zmodyfikowany token zostanie przesłany w żądaniu HTTP to dostaniemy wyjątek
48        // przy próbie odczytu jego zawartości (SignatureException: JWT signature does not match
49        // locally computed signature. JWT validity cannot be asserted and should not be trusted.)
50        return Jwts.parser()
51            .verifyWith(secretKey)
52            .build()
53            .parseSignedClaims(token)
54            .getPayload();
55    }
56
57    // Ostatnim parametrem może być referencja do metody np. Claims::getSubject
58    private <T> T extractClaim(String token, SecretKey secretKey,
59                               Function<Claims, T> claimsResolver) {
60        final Claims claims = extractAllClaims(token, secretKey);
61        return claimsResolver.apply(claims);
62    }
63
64    private String extractUserName(String token, SecretKey secretKey) {
65        return extractClaim(token, secretKey, Claims::getSubject);
66    }
67
68    public String extractUserNameFromAccessToken(String token) {
69        return extractUserName(token, this.accessSecretKey);
70    }
71
72    public String extractUserNameFromRefreshToken(String token) {
73        return extractUserName(token, this.refreshSecretKey);
74    }
75
76    private boolean isTokenValid(String token, SecretKey secretKey, UserDetails userDetails) {
77        final Claims claims = extractAllClaims(token, secretKey);
78        final Date expiration = claims.getExpiration();
79        final String userName = claims.getSubject();
80        boolean valid = !expiration.before(Date.from(Instant.now()));
81        return userName.equals(userDetails.getUsername()) && valid;
82    }
83
84 }

```

Screen 6 - Fragment kodu klasy JwtService.java w Eclipse IDE

Klasa `JwtAuthFilter` w pakiecie `com.project.config` dziedziczy po `OncePerRequestFilter`, stosując filtr jednokrotnie dla każdego przychodzącego żądania HTTP. Filtr pozyskuje nagłówek 'Authorization'. Jeśli nagłówek istnieje i zaczyna się od przedrostka 'Bearer ', ten prefiks jest ekstrahowany, a token jest przekazywany do `JwtService`.

Następnie z tokenu wyciągany jest adres email użytkownika. Jeśli użytkownik nie jest jeszcze uwierzytniony w `SecurityContextHolder`, filtr pobiera obiekty `UserDetails` z bazy za pomocą `UserDetailsService`, sprawdza czy token jest ważny i tworzy obiekt `UsernamePasswordAuthenticationToken`. Obiekt ten jest umieszczany w `SecurityContext`, co umożliwia innym klasom dostęp do danych zidentyfikowanego użytkownika.

```
01 JwtAuthFilter.java x
02 package com.project.config;
03
04 import java.io.IOException;
05 import org.springframework.http.HttpHeaders;
06 import org.springframework.http.HttpStatus;
07 import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
08 import org.springframework.security.core.context.SecurityContextHolder;
09 import org.springframework.security.core.userdetails.UserDetails;
10 import org.springframework.security.core.userdetails.UserDetailsService;
11 import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
12 import org.springframework.stereotype.Component;
13 import org.springframework.web.filter.OncePerRequestFilter;
14 import io.jsonwebtoken.ExpiredJwtException;
15 import jakarta.servlet.FilterChain;
16 import jakarta.servlet.ServletException;
17 import jakarta.servlet.http.HttpServletRequest;
18 import jakarta.servlet.http.HttpServletResponse;
19 import lombok.RequiredArgsConstructor;
20
21 @Component
22 @RequiredArgsConstructor
23 public class JwtAuthFilter extends OncePerRequestFilter {
24     private final JwtService jwtService;
25     // automatycznie wstrzykiwany (zdefiniowany jako @Bean w SecurityConfig)
26     private final UserDetailsService userDetailsService;
27
28     // gdy przychodzi żądanie HTTP to filtr zdefiniowany w metodzie doFilterInternal(...)
29     // że przechwytuje i sprawdza, czy zawiera w nagłówku Authorization poprawny token JWT
30     @Override
31     protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
32                                     FilterChain filterChain) throws ServletException, IOException {
33         final String authHeader = request.getHeader(HttpHeaders.AUTHORIZATION); //np. Bearer eyJhb...
34         final String prefix = "Bearer ";
35         if (authHeader == null || !authHeader.startsWith(prefix)) {
36             // brak tokenu w nagłówku, tylko przetwarzanie przez kolejne filtry w łańcuchu
37             filterChain.doFilter(request, response);
38             // i przerwanie realizacji metody
39             return;
40         }
41         final String token = authHeader.substring(prefix.length());
42         try {
43             final String username = jwtService.extractUserNameFromAccessToken(token);
44             if (username != null) {
45                 //Druga część warunku sprawdza czy użytkownik po raz pierwszy jest uwierzytniany.
46                 //Zapobiega to ponownemu, niepotrzebnemu procesowi uwierzytniania, jeśli jakiś
47                 //inny filtr wcześniej w tym samym łańcuchu (request chain) już to zrobił.
48                 && SecurityContextHolder.getContext().getAuthentication() == null) {
49                     UserDetails userDetails = this.userDetailsService.loadUserByUsername(username);
50                     if (jwtService.isAccessTokenValid(token, userDetails)) {
51                         // Tworzenie obiektu Authentication (authToken). Klasa UsernamePasswordAuthenticationToken
52                         // ze Spring Security jest jedną z implementacji interfejsu Authentication i służy do
53                         // w reprezentowania procesu uwierzytnienia użytkownika na podstawie nazwy użytkownika
54                         // i hasła. Pierwszy parametr konstruktora reprezentuje tożsamość użytkownika, np. login
55                         // lub obiekt użytkownika załadowany z bazy danych. Drugi zawiera dane uwierzytnienia,
56                         // najczęściej jest to hasło użytkownika. Po uwierzytnieniu zazwyczaj dla bezpieczeństwa
57                         // wartość tego pola zostaje wyczyszczona. W naszym przypadku od razu przekazujemy null.
58                         // Ostatni parametr to kolekcja ról (uprawnień) przypisanych do użytkownika.
59                         UsernamePasswordAuthenticationToken authToken =
60                             new UsernamePasswordAuthenticationToken(userDetails, null,
61                                 userDetails.getAuthorities());
62                         authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
63                         // Obiekt Authentication (authToken) jest zapisywany w SecurityContext, który jest
64                         // zarządzany przez SecurityContextHolder. Dzięki temu inne komponenty aplikacji
65                         // mogą łatwo uzyskać dostęp do danych uwierzytnienia.
66                         SecurityContextHolder.getContext().setAuthentication(authToken);
67                         // SecurityContextHolder udostępnia dane o uwierzytnionym użytkowniku i
68                         // w obrębie jednego wątku zawsze zwróci ten sam SecurityContext. Domyślnie
69                         // używa obiektu ThreadLocal do przechowywania kontekstu bezpieczeństwa,
70                         // co oznacza, że kontekst bezpieczeństwa jest zawsze dostępny dla metod w
71                         // tym samym wątku wykonania.
72                     }
73                 } catch (ExpiredJwtException e) {
74                     response.setStatus(HttpStatus.UNAUTHORIZED.value());
75                     return;
76                 }
77             // o tym trzeba pamiętać, aby umożliwić przetwarzanie przez kolejne filtry w łańcuchu
78             filterChain.doFilter(request, response);
79         }
80     }
81 }
```

Screen 7 - Kod klasy `JwtAuthFilter.java` w Eclipse IDE

W klasie SecurityWebConfig z adnotacjami @Configuration oraz @EnableWebSecurity zawarłem konfigurację bezpieczeństwa. Metoda securityFilterChain definiuje łańcuch filtrów:

- Wyłącza ochronę CSRF ze względu na architekturę bezstanową.
- Ustawia politykę sesji na STATELESS.
- Zezwala na publiczny dostęp, przez permitAll(), do punktów końcowych rejestracji, logowania i odświeżania tokenów oraz zasobów Swagger UI/OpenAPI.
- Ogranicza dostęp do punktów końcowych zarządzania studentami (/api/studenci/**), wyłącznie dla użytkowników posiadających uprawnienie 'ADMIN'.
- Wprowadza wymóg autoryzacji dla wszystkich pozostałych żądań REST API.
- Rejestruje filtr JwtAuthFilter przed filtrem domyślnym UsernamePasswordAuthenticationFilter.

```
1 package com.project.config;
2
3 import static org.springframework.security.config.Customizer.withDefaults;
4 import java.util.Arrays;
5 import java.util.Collections;
6 import org.springframework.context.annotation.Bean;
7 import org.springframework.context.annotation.Configuration;
8 import org.springframework.http.HttpHeaders;
9 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
10 import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
11 import org.springframework.security.config.http.SessionCreationPolicy;
12 import org.springframework.security.web.SecurityFilterChain;
13 import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
14 import org.springframework.web.cors.CorsConfiguration;
15 import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
16 import org.springframework.web.filter.CorsFilter;
17 import lombok.RequiredArgsConstructor;
18
19 @Configuration
20 @EnableWebSecurity
21 @RequiredArgsConstructor
22 public class SecurityWebConfig {
23
24     private final JwtAuthFilter jwtAuthFilter;
25
26     @Bean
27     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
28         return http
29             .cors(withDefaults())
30             .csrf(csrf -> csrf.disable()) // cross-site request forgery
31             .authorizeHttpRequests(auth -> auth
32                 .requestMatchers(
33                     "/api/register",
34                     "/api/login",
35                     "/api/refresh", // Kolejne adresy tego bloku powiązane są ze
36                     "/v2/api-docs", // Swaggerem (OpenAPI), przeznaczone tylko do
37                     "/v3/api-docs", // testów. Jeśli Swagger jest dostępny
38                     "/v3/api-docs/**", // publicznie, może ujawniać szczegóły API
39                     "/swagger-resources", // osobom nieuprawnionym!
40                     "/swagger-resources/**",
41                     "/configuration/ui",
42                     "/configuration/security",
43                     "/swagger-ui/**",
44                     "/webjars/**",
45                     "/swagger-ui.html").permitAll()
46                 .requestMatchers("/api/studenci/**").hasAuthority("ADMIN")
47                 .anyRequest().authenticated())
48             .sessionManagement(session -> session
49                 .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
50             .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)
51             .build();
52     }
53
54     @Bean
55     public CorsFilter corsFilter() {
56         final UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
57         final CorsConfiguration config = new CorsConfiguration();
58         config.setAllowCredentials(true);
59         // --> TODO Wpisz właściwy adres
60         config.setAllowedOrigins(Collections.singletonList("http://localhost:8081")); // --
61         config.setAllowedHeaders(Arrays.asList(
62             HttpHeaders.ORIGIN,
63             HttpHeaders.CONTENT_TYPE,
64             HttpHeaders.ACCEPT,
65             HttpHeaders.AUTHORIZATION
66         ));
67         config.setAllowedMethods(Arrays.asList(
68             "GET",
69             "POST",
70             "DELETE",
71             "PUT",
72             "PATCH"
73         ));
74         source.registerCorsConfiguration("/**", config);
75         return new CorsFilter(source);
76     }
77 }
```

Screen 8 - Kod klasy SecurityWebConfig.java w Eclipse IDE

W zakresie danych uwierzytelniających utworzyłem dwa rekordy w pakiecie com.project.auth: Credentials oraz Tokens. Metoda register(Student student) w klasie AuthServiceImpl koduje podane hasło, przypisuje nowemu użytkownikowi rolę 'USER' z bazy danych i zapisuje go z użyciem StudentService.

Metoda authenticate(Credentials credentials) wykorzystuje obiekt AuthenticationManager do weryfikacji poświadczeń. Jeśli e-mail lub hasło są niepoprawne, rzucony jest wyjątek AuthenticationException. W przypadku sukcesu dane użytkownika są pobierane, a następnie generowana jest nowa para tokenów.

Metoda refreshTokens(Tokens tokens) przyjmuje jako parametr token odświeżania, weryfikuje jego podpis i czas ważności, po czym generuje i zwraca nową parę tokenów.

```
@AuthService.java
1 package com.project.auth;
2
3 import java.util.Set;
4
5
6
7
8 @Service
9 @RequiredArgsConstructor
10 public class AuthServiceImpl implements AuthService {
11     private final StudentService studentService;
12     private final RoleRepository roleRepository;
13     private final PasswordEncoder passwordEncoder;
14     private final JwtService jwtService;
15     private final AuthenticationManager authenticationManager;
16
17     @Override
18     public void register(Student student) {
19         student.setPassword(passwordEncoder.encode(student.getPassword()));
20         var role = roleRepository
21             .findByName("USER")
22             .orElseThrow(() -> new IllegalStateException("Role USER was not initiated"));
23         student.setRoles(Set.of(role));
24         studentService.setStudent(student);
25     }
26
27     @Override
28     public Tokens authenticate(Credentials credentials) {
29         return authenticate(credentials.email(), credentials.password());
30     }
31
32     private Tokens authenticate(@NonNull String email, @NonNull String password) {
33         // Uwierzytelnianie użytkownika na podstawie podanego adresu e-mail i hasła,
34         // poprzez delegację weryfikacji do komponentu AuthenticationManager
35         // skonfigurowanego w kontekście Spring Security.
36         // Poniższa metoda sprawdza, czy poświadczenia są poprawne. Jeśli login lub hasło
37         // są błędne, metoda ta rzuci wyjątek (AuthenticationException)
38         authenticationManager
39             .authenticate(new UsernamePasswordAuthenticationToken(email, password));
40
41         // Pobranie obiektu użytkownika (już po potwierdzeniu tożsamości), aby móc wygenerować
42         // dla niego tokeny
43         var user = studentService
44             .findByEmail(email)
45             .orElseThrow(() ->
46                 new UsernameNotFoundException(String.format("User '%s' not found!", email)));
47         var accessToken = jwtService.generateAccessToken(user);
48         var refreshToken = jwtService.generateRefreshToken(user);
49         return Tokens.builder()
50             .accessToken(accessToken)
51             .refreshToken(refreshToken)
52             .build();
53     }
54
55     @Override
56     public Tokens refreshTokens(Tokens tokens) {
57         final String refreshToken = tokens.refreshToken();
58         if (refreshToken == null || refreshToken.isBlank())
59             throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Brak tokenu odświeżania");
60         final String prefix = "Bearer ";
61         final String token = refreshToken.startsWith(prefix) ?
62             refreshToken.substring(prefix.length()) : refreshToken;
63         final String email = jwtService.extractUserNameFromRefreshToken(token);
64         if (email == null || email.isBlank())
65             throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
66                 "Niepoprawny format tokenu odświeżania");
67         var user = studentService.findByEmail(email)
68             .orElseThrow(() -> new UsernameNotFoundException(
69                 String.format("User '%s' not found!", email)));
70         if (!jwtService.isRefreshTokenValid(token, user))
71             throw new ResponseStatusException(HttpStatus.UNAUTHORIZED,
72                 "Token odświeżania stracił ważność");
73         var accessToken = jwtService.generateAccessToken(user);
74         var newRefreshToken = jwtService.generateRefreshToken(user);
75         return Tokens.builder()
76             .accessToken(accessToken)
77             .refreshToken(newRefreshToken)
78             .build();
79     }
80 }

AuthController.java
1 package com.project.auth;
2
3 import org.springframework.http.HttpStatus;
4
5
6
7 @RestController
8 @RequiredArgsConstructor
9 @RequestMapping("/api")
10 @Tag(name = "Auth")
11 public class AuthController {
12     private final AuthService authService;
13     private final ValidationService<Student> studentValidator;
14
15     @PostMapping("/register")
16     public ResponseEntity<Void> register(@RequestBody Student student) {
17         studentValidator.validate(student);
18         authService.register(student);
19         return new ResponseEntity<Void>(HttpStatus.OK);
20     }
21
22     //Sprawdź zawartość zwracanego tokenu np. na https://jwt.io/
23     @PostMapping("/login")
24     public ResponseEntity<Tokens> login(@RequestBody Credentials credentials) {
25         return ResponseEntity.ok(authService.authenticate(credentials));
26     }
27
28     @PostMapping("/refresh")
29     public ResponseEntity<Tokens> refreshToken(@RequestBody Tokens tokens) {
30         return ResponseEntity.ok(authService.refreshTokens(tokens));
31     }
32 }
```

Screen 9 - Kod klas AuthServiceImpl.java i AuthController.java w środowisku Eclipse IDE

2.5. Serializacja stronicowanych wyników z PagedModel oraz obsługa autoryzacji w Swagger UI

W pakiecie `com.project.config` utworzyłem klasę konfiguracyjną `SwaggerConfig`, aby umożliwić testowanie chronionych punktów końcowych usługi REST z poziomu Swagger UI. W metodzie `customOpenAPI` jest definiowany schemat bezpieczeństwa o nazwie `'bearerAuth'` typu HTTP z `bearerFormat = JWT`. Dzięki temu w nagłówku interfejsu Swagger pojawił się przycisk `'Authorize'`, umożliwiający wklejenie tokena dostępowego, który jest automatycznie dołączany do każdego wysyłanego zapytania.

Podczas zwracania stronicowanych wyników zapytania z bazy danych Spring Boot domyślnie serializuje obiekty klasy `PageImpl`. Problem pojawiających się w konsoli ostrzeżeń, wskazujących na ryzyko niestabilności struktury JSON w przyszłych wersjach frameworka, rozwiązałem poprzez utworzenie w pakiecie `com.project.config` klasy konfiguracyjnej `JacksonConfig` z adnotacją `@EnableSpringDataWebSupport(pageSerializationMode =`

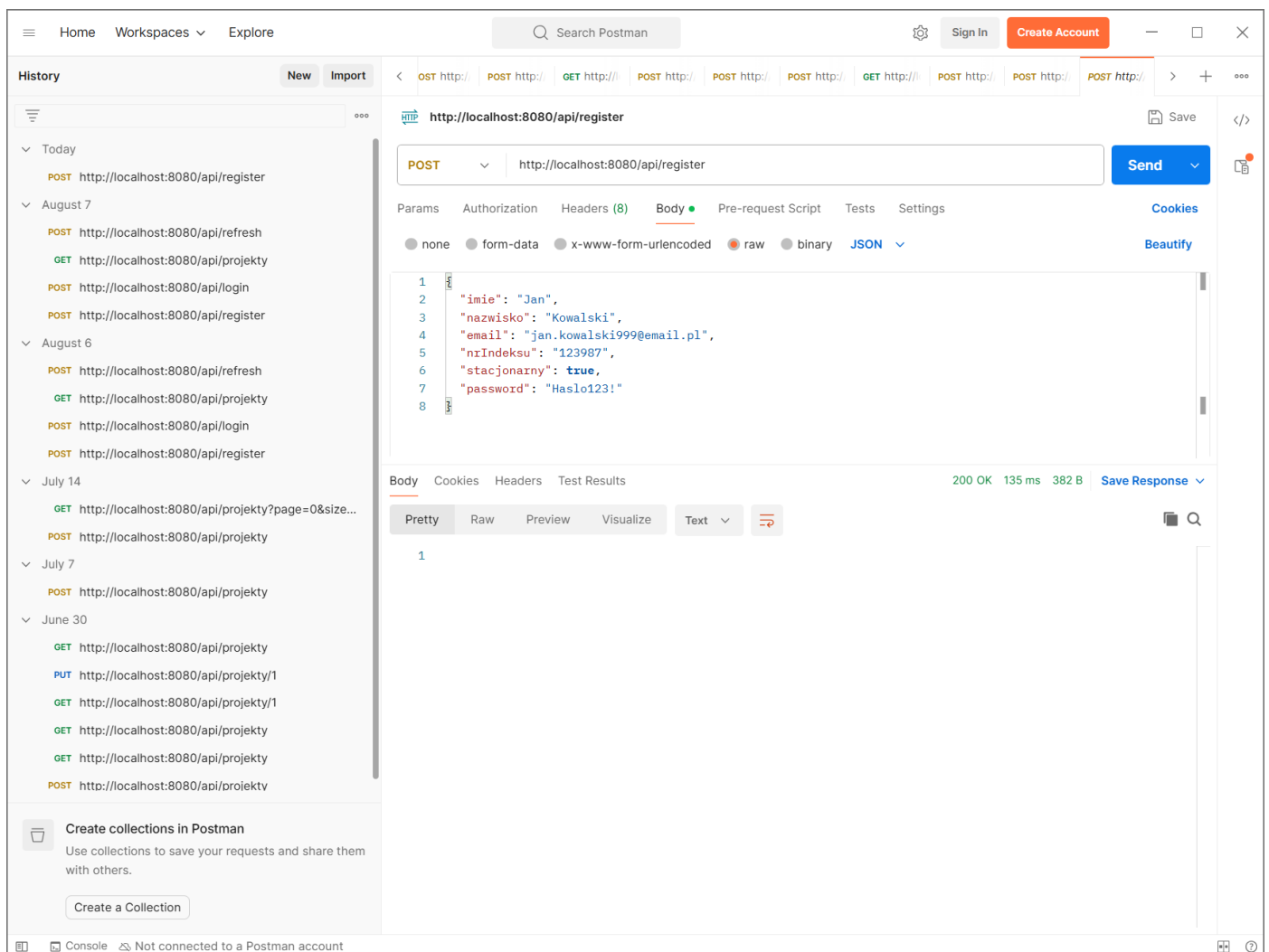
`EnableSpringDataWebSupport.PageSerializationMode.VIA_DTO)`

Ta adnotacja pozwala pogrupować metadane paginacji w zagnieżdżonym obiekcie `'page'`, oddzielnym od tablicy danych `'content'`.

2.6. Testowanie zabezpieczonej usługi REST API w programie Postman oraz w Swagger UI

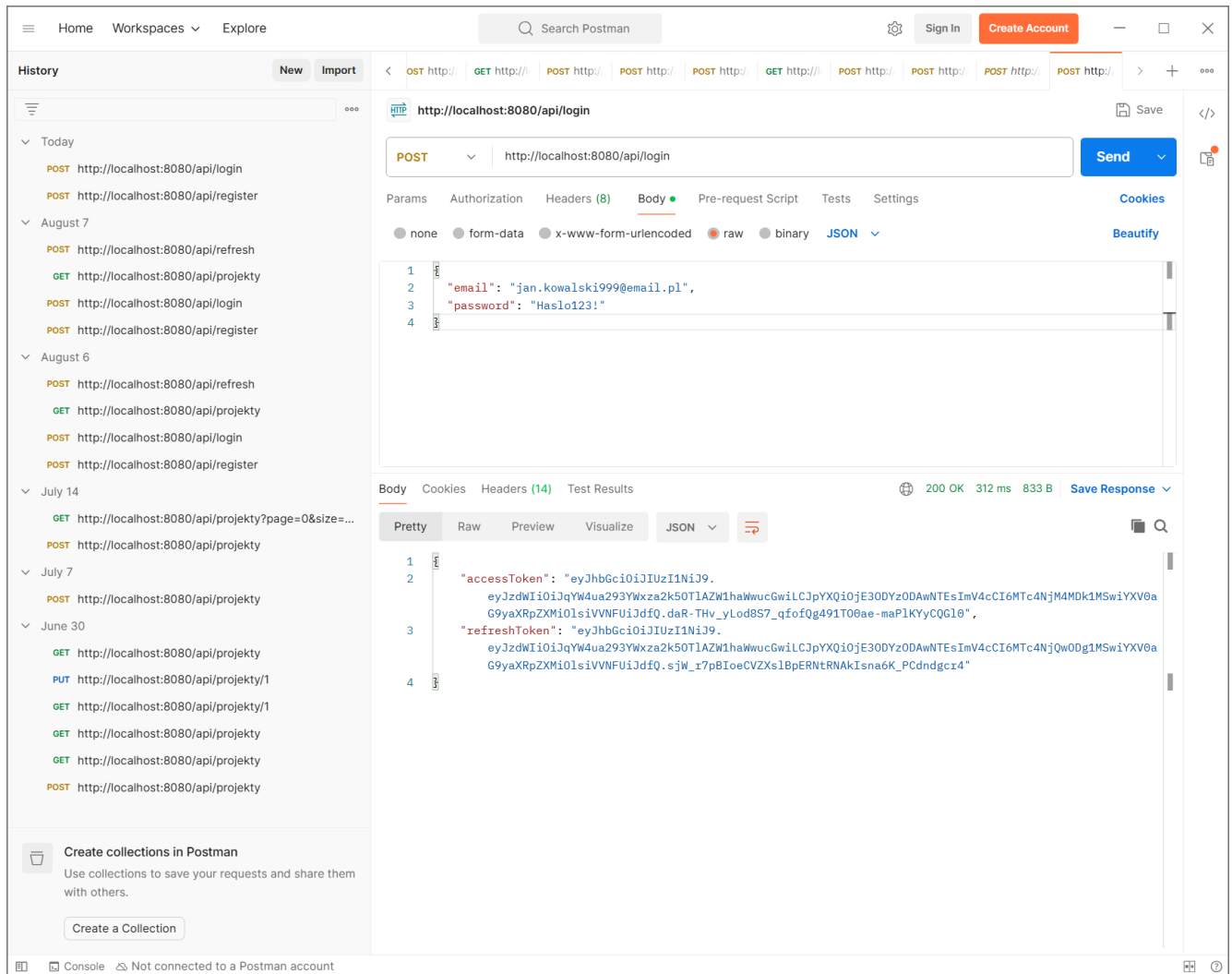
Po skompilowaniu i uruchomieniu aplikacji na porcie 8080 przeprowadziłem testy autoryzacyjne w Postman oraz Swagger UI:

1) Rejestracja nowego użytkownika: Żądanie POST pod adres `http://localhost:8080/api/register` z danymi nowego studenta (imię, nazwisko, email, nrIndeksu, password) w JSON. Serwer zarejestrował studenta w PostgreSQL, zakodował hasło (BCrypt), przypisał rolę USER i zwrócił kod 200 OK. Przetestowałem też obsługę błędów walidacji (400 Bad Request przy niepoprawnych danych).



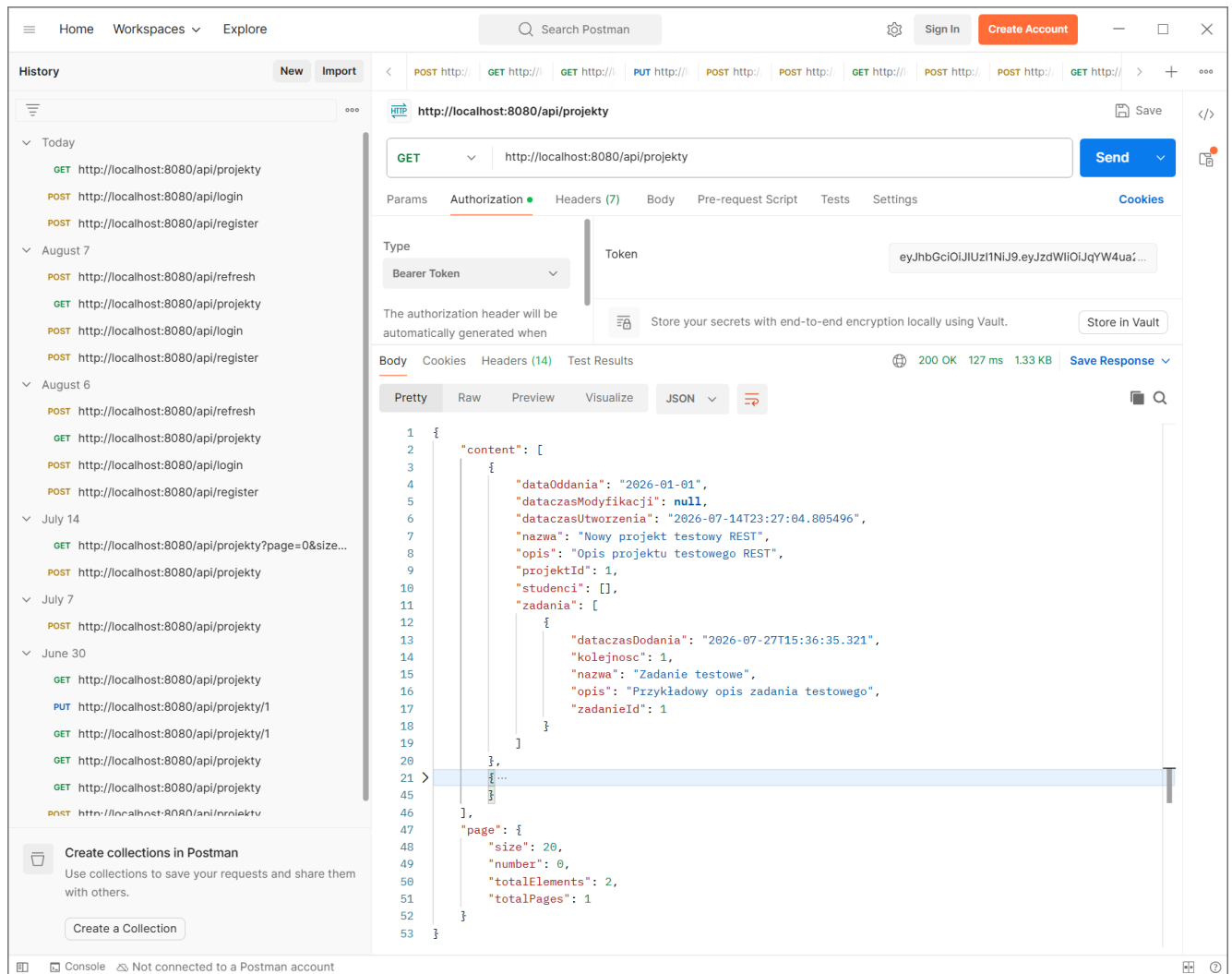
Screen 10 - Rejestracja nowego użytkownika w programie Postman

2) Logowanie i generowanie tokenów: Żądanie POST pod adres `http://localhost:8080/api/login` z danymi logowania w formacie JSON (email, password). Serwer zweryfikował poświadczenia i zwrócił obiekt JSON z dwoma tokenami: `accessToken` oraz `refreshToken`.



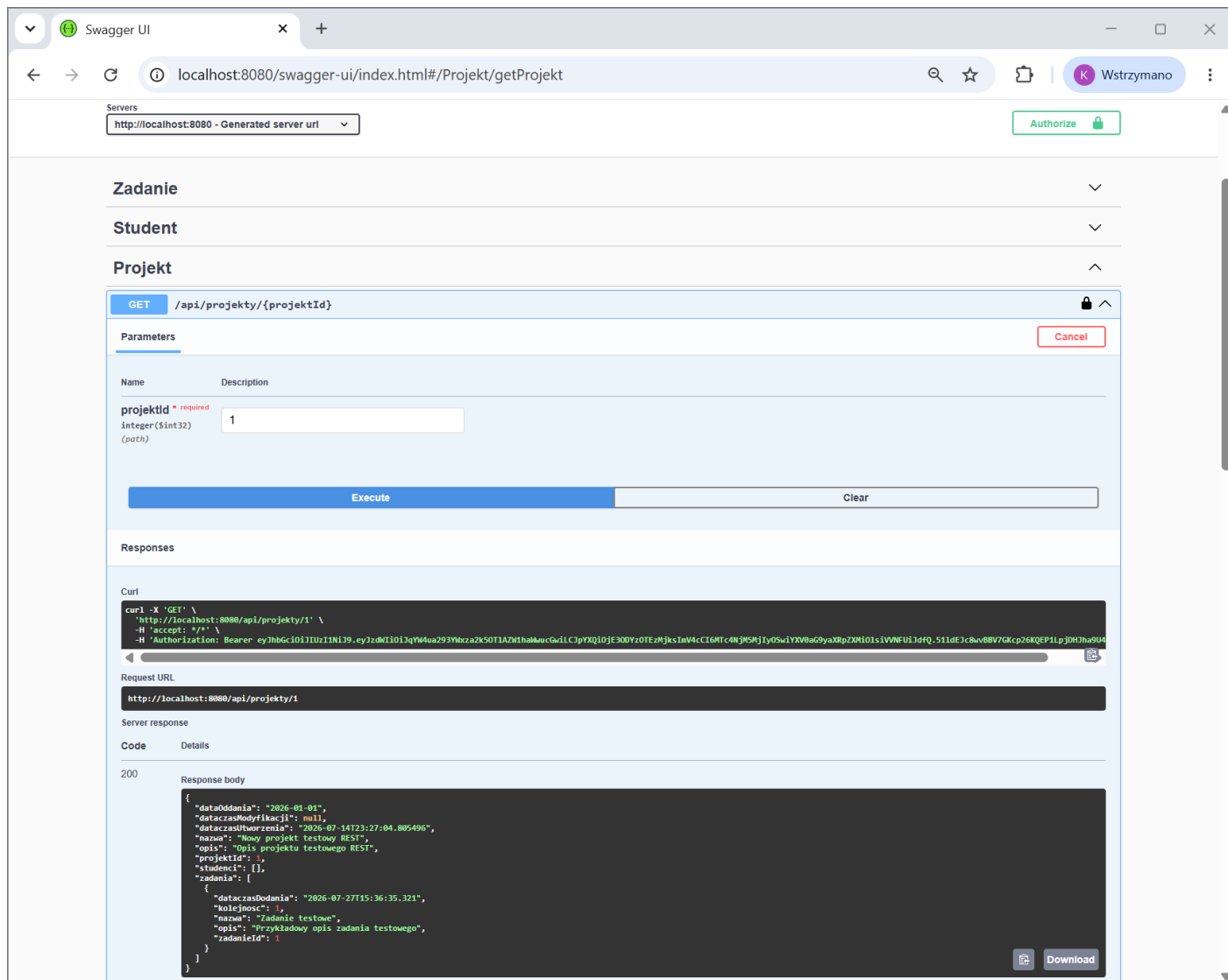
Screen 11 - Logowanie użytkownika i generowanie tokenów JWT w programie Postman

3) Autoryzacja chronionych punktów końcowych oraz obsługa PagedModel: Próba wywołania żądania GET `http://localhost:8080/api/projekty` bez przekazania tokena zakończyła się kodem 401 Unauthorized. Po wklejeniu accessTokenu w zakładce Authorization (Type: Bearer Token) serwer zwrócił listę projektów ze statusem 200 OK i zagnieżdżonym obiektem 'page'. Można również wysłać żądanie POST `/api/refresh` w celu uzyskania nowych tokenów.



Screen 12 - Dostęp do zasobu chronionego REST API przy użyciu Bearer Token w Postmanie

4) Testy w Swagger UI: W przeglądarce wszedłem pod adres <http://localhost:8080/swagger-ui/index.html>. Kliknąłem przycisk 'Authorize', po czym pojawiło się okno dialogowe, w którym wkleiłem token JWT. Po udanej autoryzacji z Bearer Token przetestowałem chronione endpointy REST API, np. GET /api/projekty, uzyskując odpowiedź ze statusem 200 OK.



Screen 13 - Testowanie przykładowego chronionego endpointa GET /api/projekty w Swagger UI po udanej autoryzacji z Bearer Token

3. Wnioski końcowe/Podsumowanie

Realizacja siódmego laboratorium pozwoliła zabezpieczyć usługę sieciową typu REST w oparciu o technologię JWT (JSON Web Token). Zastosowana w jego ramach architektura bezstanowa jest skalowalna i bardzo wydajna. Dzięki rezygnacji z tradycyjnych sesji, serwer http nie przechowuje informacji o aktywnych, zalogowanych użytkownikach, a każda weryfikacja tożsamości odbywa się przez sprawdzenie cyfrowego podpisu.

Zastosowanie mechanizmu dwóch tokenów - krótkotrwałego Access Token (ważnego przez 15 minut) oraz długotrwałego Refresh Token (ważnego przez 8 godzin) - pozwoliło połączyć wysoki poziom bezpieczeństwa z wygodą użytkowników. Krótki czas ważności tokena dostępowego utrudnia ewentualne próby przechwycenia poświadczeń w niepowołany sposób. Z kolei długi czas tokena odświeżania sprawia, że nie ma potrzeby ponownego podawania hasła przy odnawianiu uprawnień. Wykorzystanie szyfrowania BCrypt z kryptograficzną solą oraz mechanizmu kontroli dostępu opartego na rolach (RBAC) z podziałem na uprawnienia USER i ADMIN, również przyczyniają się do podniesienia poziomu bezpieczeństwa zasobów w bazie danych.

Uważam zaimplementowanie globalnej obsługi błędów i walidacji wejściowej za bardzo przydatne uzupełnienie dotychczasowego projektu. Adnotacja `@RestControllerAdvice` w połączeniu z `ValidationService` oraz obiektem `Violation` zapewnia spójną strukturę komunikatów błędów (np. HTTP 400 Bad Request) w całym API. Natomiast zastosowanie konfiguracji `VIA_DTO` rozwiązało problem ostrzeżeń o niestabilnej serializacji klasy `PagedListImpl` i zapewniło czytelną strukturę danych stronicowanych w odpowiedziach JSON w standardzie `PagedModel`.